

**CENTRALE
LYON**

PAR N°163

Génération de musique rock par réseau de neurones

Élèves :

Alexandre MASSIES
Germain DAL

Enseignants :

Mohsen ARDABILIAN
Alexandre SAIDI

8 avril 2025

Table des matières

Introduction	3
1 État de l'art	4
1.1 La créativité computationnelle	4
1.1.1 Définition et enjeux généraux	4
1.1.2 Applications en musique générative	4
1.2 Encodage de la musique	5
1.2.1 Génération audio et Génération symbolique	5
1.2.2 Formats symboliques utilisés en musique	6
1.3 Réseaux de neurones appliqués à la musique	7
1.3.1 Principes généraux des réseaux de neurones artificiels	7
1.3.2 Architectures spécifiques adaptées à la génération musicale	8
1.3.3 Entraînement	10
1.4 Décodage de la musique générée	11
1.4.1 Conversion au format MIDI	11
1.4.2 Transformation au format audio	11
2 Analyse du contexte musical	12
2.1 Notions fondamentales de théorie musicale	12
2.1.1 Gammes, notes et fréquences	12
2.1.2 Structure harmonique : accords et progressions harmoniques typiques	12
2.2 Particularités stylistiques de la musique Rock	13
2.2.1 Instrumentation typique du rock	13
2.2.2 Progressions harmoniques courantes du rock	13
2.2.3 Structures rythmiques typiques (mesures, tempo, BPM)	14
2.3 Représentation MIDI approfondie	14
2.3.1 Structure technique des fichiers MIDI	14
2.3.2 Encodage des événements musicaux (Note-On/Note-Off, Pitch, Velocity)	14
3 Modélisation par réseaux de neurones récurrents (RNN)	15
3.1 Architecture du modèle utilisé	15
3.1.1 Présentation détaillée de l'architecture LTMS	15
3.2 Implémentation technique	17
3.2.1 Environnement de développement et bibliothèques utilisées	17
3.2.2 Organisation du code source	17
4 Méthode d'entraînement du modèle	18
4.1 Préparation des données	18
4.1.1 Sélection et description de la base de données MIDI	18
4.1.2 Méthodes de prétraitement et encodage	18
4.1.3 Représentation finale des données pour l'entraînement	20
4.2 Paramètres et hyperparamètres d'entraînement	20
4.3 Processus détaillé d'entraînement	20



5	Génération de musique par le modèle entraîné	21
5.1	Méthodologie de génération musicale	21
5.2	Décodage des résultats en fichiers musicaux	22
6	Résultats et analyse	23
6.1	Génération de progressions harmoniques avec LSTM	23
6.1.1	Principe et tests réalisés	23
6.1.2	Évaluation qualitative des séquences générées	23
6.2	Génération de morceaux complets	24
6.2.1	Présentation de l'interface graphique utilisateur	24
6.2.2	Analyse qualitative des résultats	25
6.3	Limites et perspectives	27
	Conclusion	28
	Bibliographie	29

Résumé

Ce Projet d'Application orienté recherche a pris pour sujet la génération de musique par réseaux de neurones. Le but du projet était d'obtenir un modèle de réseau de neurones qui permet de générer des morceaux avec plusieurs instruments, à partir d'une base de données de musique rock.

Le présent rapport détaille tout d'abord l'objectif et le contexte du projet. On revient ensuite sur un état de l'art qui a permis d'identifier les différentes possibilités pour le modèle. Le rapport détaille aussi rapidement le contexte musical. On présente ensuite l'architecture du modèle retenu, et la méthode d'entraînement prévue. La musique est encodée en langage Python pour être utilisable par un réseau de neurones, à partir de fichiers MIDI. Quelques résultats sur des données d'entrées de taille réduites sont présentés.

Ceci permet de conclure quant à l'avancement du projet et à l'atteinte de l'objectif.

Abstract

The subject of this research-oriented Application Project is music generation by means of neural networks. The aim of the project was to obtain a neural network model that can generate music pieces with multiple instruments, from a rock music database.

This report begins by detailing the origin and objectives of the project. It then reviews the state of the art, which brought to light the various possibilities for the model. The report also details the musical context. We then present the architecture of the model chosen, and the training method planning. The music is encoded in Python language to be handled by a neural network, from MIDI files. Results of generated music based on reduced-size inputs are discussed.

This allows conclusions to be drawn about the progress of the project and whether the objective has been met.

Introduction générale

Ces dernières années, l'intelligence artificielle (IA) a profondément modifié le paysage technologique, ouvrant de nouvelles voies dans de nombreux domaines d'application. Parmi ces avancées, les modèles génératifs occupent une position particulière, étant capables de produire des contenus originaux tels que des images (cf. 1), des textes ou encore des séquences musicales. Ce champ spécifique, connu sous le nom de créativité computationnelle, repousse les frontières traditionnelles des capacités des systèmes informatiques, permettant ainsi d'explorer des processus de création jusque-là réservés aux êtres humains, avec l'aide d'entités comparables à des systèmes de boîte noire.

La musique se révèle particulièrement propice à l'application de ces techniques, car elle allie rigueur formelle et liberté créative. Le genre rock, en particulier, présente un intérêt significatif en raison de ses caractéristiques rythmiques marquées, ses structures harmoniques distinctives, et la variété instrumentale qu'il mobilise. Ainsi, ce projet vise à explorer et à démontrer la faisabilité de générer automatiquement des compositions musicales dans le style rock à l'aide de réseaux de neurones récurrents.

La problématique centrale abordée est la suivante : **Comment mettre en place une IA basée sur des réseaux de neurones récurrents pour générer de la musique dans un style rock ?**

Pour répondre à cette problématique, le modèle proposé repose sur une architecture spécifique de réseaux neuronaux, dite LTMS (Local Time Multi-Scale), capable de gérer simultanément plusieurs instruments de façon coordonnée et cohérente. Le projet comprend une implémentation complète, depuis l'acquisition et le prétraitement des données musicales jusqu'à l'entraînement du modèle et la génération des morceaux musicaux finaux, accessible via une interface graphique utilisateur.

Ce rapport débute par une présentation détaillée de l'état de l'art relatif aux techniques de génération musicale par réseaux de neurones. Il s'attarde ensuite sur les spécificités musicales inhérentes au genre rock, puis explicite la démarche méthodologique employée, ainsi que l'architecture choisie. Enfin, une analyse approfondie des résultats obtenus est proposée, accompagnée de perspectives pour de futures améliorations.



FIGURE 1 – Illustration réalisée par IA inspirée de *Wish You Were Here* de Pink Floyd

1 État de l'art

1.1 La créativité computationnelle

1.1.1 Définition et enjeux généraux

La créativité computationnelle [1] consiste en la capacité d'un système informatique à produire des résultats qui peuvent être jugés comme créatifs par les standards humains. Ce domaine de recherche se situe à l'intersection entre l'intelligence artificielle, les sciences cognitives et l'informatique théorique, visant à reproduire voire dépasser les capacités créatives humaines. Les enjeux sont variés : ils touchent autant aux aspects éthiques, notamment la reconnaissance de l'originalité et les problématiques de propriété intellectuelle, qu'aux défis techniques tels que la cohérence et la pertinence des contenus générés.

1.1.2 Applications en musique générative

Dans le domaine musical, les applications de créativité computationnelle reposent principalement sur l'utilisation d'algorithmes issus de l'intelligence artificielle pour produire

des œuvres originales. On distingue principalement les approches fondées sur des systèmes algorithmiques précis et celles s'appuyant sur l'apprentissage automatique via des réseaux de neurones. Ces systèmes trouvent des applications concrètes variées : assistance à la composition, génération dynamique de bandes sonores interactives notamment dans les jeux vidéo, ou encore production automatisée de musiques d'ambiance adaptées à différents contextes (voir le tableau 1).

A ce jour, de nombreux logiciels de composition disposent d'outils basés sur l'intelligence artificielle pour assister l'utilisateur, comme *Orb Composer*, *SongR*, *Riffusion* et bien d'autres. Ces logiciels connaissent une popularité grandissante, accompagnée d'améliorations régulières.

Application	Description	Exemple
Assistance à la composition	Aider les compositeurs à créer de nouvelles œuvres musicales grâce à l'IA	AIVA [2] (composition symphonique)
Bandes sonores interactives	Génération dynamique de musique adaptée aux actions de l'utilisateur, notamment dans les jeux vidéo	Melodia, systèmes temps réel
Production automatisée de musiques	Création de musiques d'ambiance ou d'accompagnement pour divers contenus (podcasts, vidéos, etc.)	Soundful, plateformes en ligne

TABLE 1 – Applications clés de la musique générative par IA

1.2 Encodage de la musique

1.2.1 Génération audio et Génération symbolique

Pour la génération de musique par intelligence artificielle, deux domaines distincts prédominent : la musique audio, et la musique symbolique. Dans le premier cas, l'enjeu est de traiter des données sous la formes de canaux de signaux en fonction du temps, donc depuis des fichiers .mp3 ou .wav. Ce domaine fait intervenir des outils de traitement du signal et de génération de signaux. La génération symbolique, elle repose sur l'analyse de données sous formes de partitions ou équivalents, où sont recensées des informations comme le rythme, les notes jouées, le tempo, la tonalité...

L'avantage de la génération audio, par rapport à une analyse symbolique, est qu'elle est plus efficace lorsqu'il s'agit de créer des timbres de sons complexes, et une réelle expressivité de la musique. En revanche, il est difficile de construire de longs modèles structurels grâce à cette approche.

La génération symbolique, elle, a la vertu de cerner remarquablement aisément les motifs et constructions musicales complexes. Cette approche est plus à même de générer des structures cohérentes plus longues. Son principal désavantage est le manque d'expressivité et de dynamique.

C'est cette deuxième approche qui sera privilégiée pour le projet. Les formats présentés ensuite sont différentes représentations symboliques de la musique.

1.2.2 Formats symboliques utilisés en musique

Pour représenter symboliquement la musique, il est nécessaire de s'appuyer sur des standards d'encodage [3]. Parmi les formats symboliques couramment utilisés en intelligence artificielle musicale, on retrouve principalement le MIDI, le Pianoroll, et la notation ABC, qui présentent leurs avantages et inconvénients (cf 2).

MIDI

Le format MIDI (Musical instrument digital interface) est un format standard de représentation symbolique par événements. Le format répandu à une échelle industrielle, utilisé par les musiciens et les producteurs. Il existe donc beaucoup d'outils pour le manipuler facilement. Les événements basiques sont par exemple *Note-on*, *Note-off*, en spécifiant le ton, la vélocité, etc. En ajoutant plusieurs canaux en parallèle, on peut représenter plusieurs instruments. C'est ce qui est utilisé par le modèle BandNet [4] pour les musiques dans le style des Beatles par exemple.

Toutefois, certaines nuances trop rapides sont difficiles à représenter puisque le temps est discrétisé. Les détails d'interprétation ne sont pas non plus capturés par ce format.

Pianoroll

Ce format utilise une représentation matricielle où chaque dimension encode les notes jouées (hauteur) et leur durée (temps), comme pour un piano automatique. L'avantage est que cela ressemble beaucoup à une partition visuelle, cela peut presque être traité comme une image. C'est bien compatible avec les modèles de Deep Learning. Pour avoir plusieurs instruments, on superpose plusieurs matrices.

D'un autre côté, le stockage nécessaire est très grand si le morceau a de longues séquences. Mais il n'y a pas d'informations sur les notes jouées : on ne distingue pas une note longue et des notes courtes répétées.

Notation ABC

On peut aussi simplement utiliser un format texte. La représentation la plus répandue est la notation ABC. Un en-tête précise le titre, la signature temporelle, la clé... Le corps de texte comprend ensuite les notes (avec la lettre correspondante) et les barres de mesures. La simplicité du format fait qu'il est supporté par beaucoup d'outils.

Mais cela ne représente qu'un instrument à la fois, et n'ajoute pas d'information sur les notes jouées.

Critère	Audio	MIDI	Pianoroll	Texte (ABC)
Expressivité	Élevée	Moyenne	Faible	Faible
Facilité d'utilisation	Faible	Élevée	Moyenne	Élevée
Efficacité mémoire	Faible	Élevée	Faible	Élevée
Complexité temporelle	Moyenne	Moyenne	Élevée	Faible
Représentation multi-instruments	Oui	Oui	Oui	Non

TABLE 2 – Comparaison des représentations symboliques et audio de la musique

1.3 Réseaux de neurones appliqués à la musique

1.3.1 Principes généraux des réseaux de neurones artificiels

Les réseaux de neurones artificiels s'inspirent du cerveau humain, où des impulsions électriques circulent entre les neurones via des synapses. Chaque neurone biologique (cf. 2) reçoit des entrées par ses dendrites, les traite via son noyau et son axone, puis transmet une sortie. Cependant, la modélisation artificielle reste simplifiée, car le cerveau humain dépasse largement les capacités des ordinateurs actuels. Les réseaux artificiels offrent ainsi une approximation utile mais limitée, illustrée par le perceptron (cf. 3).

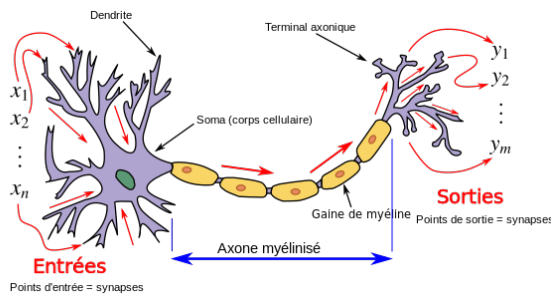


FIGURE 2 – Schéma d'un neurone humain [5]

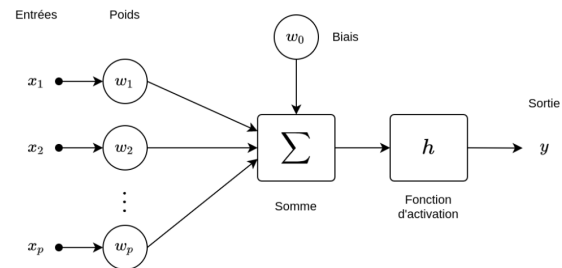


FIGURE 3 – Schéma de principe du perceptron [5]

Le perceptron est la brique élémentaire des réseaux de neurones. Il reçoit un vecteur d'entrée $(x_1, x_2, \dots, x_p)$, applique une fonction d'agrégation $s = w_0 + \sum_{i=1}^p w_i x_i$, où w_i sont des poids et w_0 un biais, puis une fonction d'activation souvent non linéaire. La sortie résultante, y , est une transformation des entrées pondérées, représentant l'information traitée par le perceptron.

Ces neurones s'organisent en couches successives formant un réseau capable d'apprendre des représentations complexes à partir des données fournies (cf. 4).

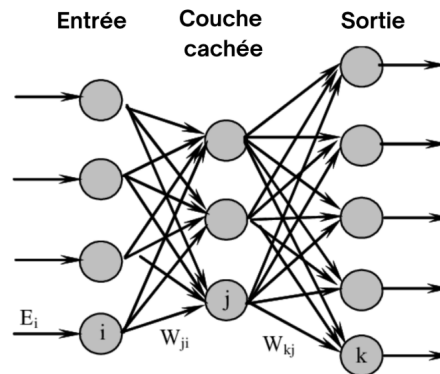


FIGURE 4 – Schéma d'un réseau de neurones simple

Ce schéma singe le fonctionnement de l'organe humain à une échelle moindre, tout de même (cf. 3). Pour un ordre d'idée, bien que le cerveau aie environ 80 milliards de neurones [6], un modèle actuel comme GPT-3 peut atteindre 175 milliards de neurones [7].

Aspect	Cerveau humain	Modèle IA (ex : GPT-3)
Nombre de "neurones"	~80 milliards	Pas de neurones, mais ~175 milliards de poids (paramètres)
Connexions (synapses ou poids)	~ 10^{15} (1 quadrillion)	~ 10^{11} à 10^{12}
Capacité mémoire estimée	~2.5 Po (pétaoctets)	~700 Go (en FP32)
Consommation énergétique	~20 W	Plusieurs mégawatts (entraînement)
Poids approximatif	~1.4 kg	Stocké sur disque, pas physique (700 Go $\approx$ 0.7 kg sur SSD)

TABLE 3 – Comparaison à l'échelle entre le cerveau humain et un modèle de réseau de neurones artificiel

1.3.2 Architectures spécifiques adaptées à la génération musicale

Plusieurs architectures neuronales ont été spécifiquement adaptées à la génération musicale [8].

RNN (Recurrent Neural Networks)

Les réseaux de neurones récurrents (RNN) sont conçus pour traiter les séquences en modélisant les dépendances temporelles entre les éléments. Le réseau garde une mémoire, actualisée en fonction de la mémoire précédente et de l'entrée actuelle (cf. 5). Bien qu'ils souffrent de limitations comme le gradient qui disparaît, ils restent largement utilisés pour la création de mélodies et rythmes cohérents en génération musicale, où les relations séquentielles entre les notes sont essentielles.

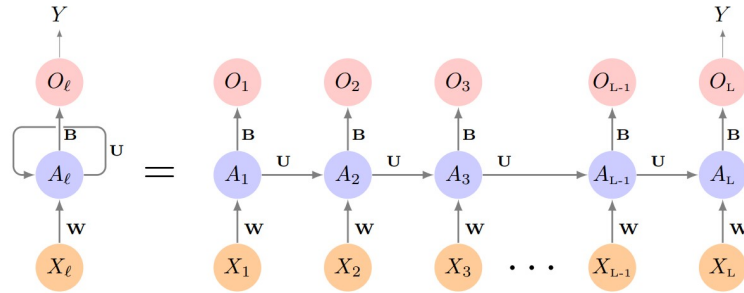


FIGURE 5 – Schéma explicatif du fonctionnement d'un réseau de neurones récurrent

LSTM (Long Short-Term Memory)

Les réseaux LSTM sont une forme avancée de réseaux de neurones récurrents conçue pour résoudre le problème de la dépendance à long terme dans les séquences. Ils incluent des cellules mémoire capables de conserver ou d'oublier des informations. Cela les rend particulièrement efficaces pour des tâches nécessitant le traitement de longues séquences, comme ici la génération de musique.

Variational Autoencoder (VAE)

Les auto-encodeurs variationnels utilisent la structure d'auto-encodeur pour générer des données similaires à l'entrée. Un auto-encodeur est un réseau qui fonctionne par compression de l'information dans un espace de plus faible dimension (couche cachée centrale), puis une couche de décodage (voir fig. 6). Ils peuvent être utilisés en apprentissage non-supervisé.

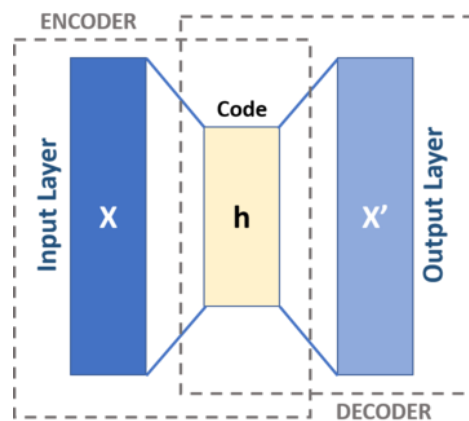


FIGURE 6 – Schéma d'un auto-encodeur

L'amélioration en VAE ajoute un côté probabiliste, permettant une génération musicale basée sur des distributions variées.

Transformers

Les Transformers utilisent un mécanisme d'attention qui capture efficacement les dépendances à longue portée dans les séquences. En musique, des modèles comme ceux de Suno AI [9] exploitent cette architecture pour générer des compositions riches et cohérentes, en modélisant à la fois les relations temporelles et contextuelles complexes.

GAN (Generative Adversarial Networks)

Les GAN reposent sur deux réseaux : un générateur et un discriminateur (voir fig. 7). Le générateur crée des échantillons, tandis que le discriminateur évalue leur authenticité par rapport à des données réelles. Les GAN sont souvent utilisés pour générer des données réalistes, comme des images ou de la musique, en améliorant progressivement la qualité des échantillons générés via un processus d'entraînement compétitif.

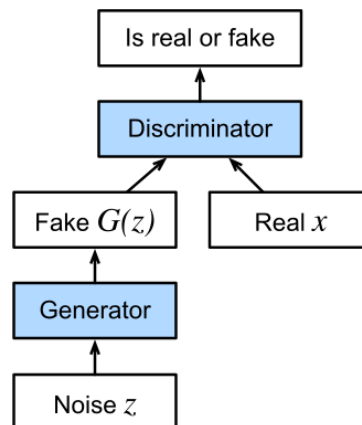


FIGURE 7 – Schéma d'un GAN

1.3.3 Entraînement

Bases de données de fichiers MIDI

Des données en format MIDI sont facilement trouvables gratuitement en ligne, et pour beaucoup de genres musicaux.

- Site d'archive Rppmf [10]. Contient les musiques de groupes de rock pop et progressif, triés par artiste et par albums.
- Base de données de musique classique MAESTRO [11]. Le jeu de données comporte environ 200 heures de performances de piano, en fichiers audio et symboliques.
- University of Rochester Multi-Modal Music Performance (URMP) Dataset [12]. Inclut des vidéos de performances de musique classique, et des fichiers MIDI correspondants.

Données et qualité du modèle

Quand il s'agit de sélectionner une base de données pour garantir le bon entraînement du modèle que l'on cherche à développer, quelques unes de ses caractéristiques doivent

répondre à des exigences.

Qualité du jeu de données Une base de données alimentée par des professionnels est importante, pour garantir notamment la justesse de l'encodage de musiques dans les fichiers MIDI, sans problèmes de transcription, sans manque de données, ni trop de simplifications des annotations. Ceci est essentiel dans le but de construire un modèle efficace et dont les prédictions sont cohérentes.

Diversité de données Il est primordial de disposer de données suffisamment différentes lorsque l'on désire avoir la certitude que le réseau de neurones sera capable de produire des sorties intéressantes, sans privilégier de façon trop évidente certaines structures rencontrées dans les datasets. Faire varier les artistes représentés est une première façon de régler ce problème, puisque chacun a une signature musicale distinctes, des manières de composer propres, des progressions ou arrangements, styles évidemment, relativement récurrents, ce qui restreint l'objectivité des données et affecte à terme l'IA mise en oeuvre.

1.4 Décodage de la musique générée

1.4.1 Conversion au format MIDI

Des modules disponibles en Python permettent de réaliser la création et la modification de fichiers MIDI. Il est alors aisé de faire la conversion de l'encodage en format MIDI, en générant tous les canaux nécessaires et en les remplissant successivement selon la symbolique adoptée pour l'encodage qui passe dans le réseau de neurones.

1.4.2 Transformation au format audio

Dans le but de pouvoir écouter les productions musicales réalisées sous format MIDI, il est possible d'utiliser des convertisseurs MIDI vers .mp3. Nous avons aussi utilisé le logiciel *MuseScore* pour synthétiser la musique et contrôler le contenu du fichier produit. Par exemple, un fichier correspondant au titre "Twist and shout" des Beatles est ouvert avec *MuseScore* en figure 8.



FIGURE 8 – Visualisation du fichier "Twist and shout" avec *MuseScore 3*

2 Analyse du contexte musical

2.1 Notions fondamentales de théorie musicale

2.1.1 Gammes, notes et fréquences

La musique occidentale standard repose sur une division en gammes composées généralement de douze notes distinctes. Ces notes se répètent par cycles appelés octaves, chaque octave doublant ou divisant par deux la fréquence sonore de référence (cf. 9). Par exemple, la note de référence, le La₄, a une fréquence standardisée de 440 Hz ; la note suivante à l'octave supérieure (La₅) aura alors une fréquence de 880 Hz. Ce système structuré permet une représentation mathématique simple des hauteurs musicales et facilite leur encodage numérique.

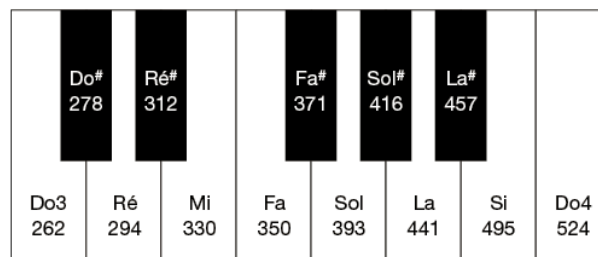


FIGURE 9 – Schéma représentant visuellement une octave avec les notes et fréquences correspondantes

2.1.2 Structure harmonique : accords et progressions harmoniques typiques

En théorie musicale, un accord désigne la superposition d'au moins trois notes jouées simultanément. Ces accords structurent la dimension harmonique de la musique et constituent la base des progressions harmoniques, séquences ordonnées d'accords formant la trame musicale d'un morceau (cf. 10). Certaines progressions harmoniques sont très fréquemment utilisées car elles procurent une stabilité sonore agréable à l'écoute, par exemple la progression I-IV-V (Do-Fa-Sol) ou la progression I-V-vi-IV (Do-Sol-La mineur-Fa).

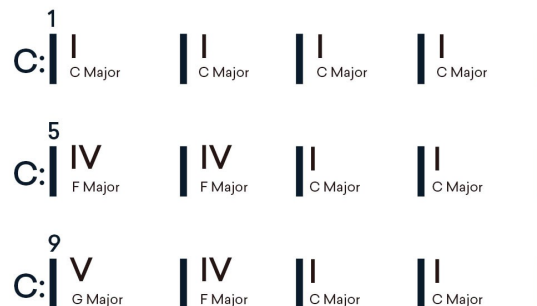


FIGURE 10 – Exemples de progressions harmoniques avec la nomenclature classique

2.2 Particularités stylistiques de la musique Rock

La musique rock présente certaines caractéristiques stylistiques spécifiques qui la distinguent d'autres genres musicaux. Ces particularités incluent notamment son instrumentation caractéristique, ses structures rythmiques marquées et ses progressions harmoniques souvent simples mais très expressives. Il s'agit d'un genre très vaste de la musique, englobant les classiques psychédéliques des Beatles et d'Hendrix, les rythmes très pop de la new wave de New Order ou Talking Heads, le rock progressif de Pink Floyd et King Crimson, le grunge de Nirvana, l'art rock de Radiohead et David Bowie, la folk de Neil Young et Bob Dylan, le hard rock mélangé au blues de Led Zeppelin, le rock alternatif de Pixies et The Strokes, le punk de The Clash, le noise rock de Sonic Youth, le post punk de Joy Division et The Cure et des genres plus exotiques comme le jazz rock de Talk Talk et Black Midi ou encore le post-rock expérimental de Swans et Slint. Néanmoins, des spécificités restent remarquables et permettent de caractériser le genre facilement.

2.2.1 Instrumentation typique du rock

L'instrumentation rock classique inclut habituellement une ou deux guitares électriques (lead et rythmique), une basse électrique, une batterie complète et souvent des claviers ou synthétiseurs (cf. 11). Ces instruments contribuent ensemble à créer le son dynamique et énergique typique de ce style.



FIGURE 11 – Yes en concert en 1971

2.2.2 Progressions harmoniques courantes du rock

Les morceaux rock utilisent souvent des progressions d'accords répétitives et mémorables, telles que les progressions en accords de puissance (power chords) composées essentiellement de fondamentales et de quintes, ou encore des séquences très courantes comme la progression I–IV–V–I ou la progression I–V–vi–IV, utilisées largement dans les ballades rock.

2.2.3 Structures rythmiques typiques (mesures, tempo, BPM)

La musique rock se caractérise souvent par des structures rythmiques simples mais très marquées, utilisant fréquemment une signature rythmique à quatre temps (4/4), comme en figure 12. Le tempo varie généralement de modéré à rapide, typiquement entre 90 et 140 battements par minute (BPM). Ce rythme clairement défini facilite l'identification immédiate et la mémorisation des morceaux.

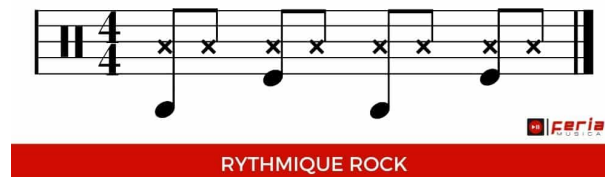


FIGURE 12 – Exemple de rythme classique de la batterie en rock

2.3 Représentation MIDI approfondie

2.3.1 Structure technique des fichiers MIDI

Le MIDI (Musical Instrument Digital Interface) constitue le standard industriel de représentation numérique des événements musicaux. Un fichier MIDI est composé de plusieurs pistes indépendantes, chacune pouvant correspondre à un instrument particulier. Les différentes pistes sont clairement visibles avec *MuseScore* en figure 8. Les informations principales encodées sont le déclenchement des notes (événements Note-On), leur arrêt (événements Note-Off), leur hauteur (pitch), leur vitesse (intensité sonore) et la temporalité précise des événements. Cette structure permet un traitement informatique précis et efficace des données musicales.

2.3.2 Encodage des événements musicaux (Note-On/Note-Off, Pitch, Velocity)

Dans un fichier MIDI, chaque événement musical est précisément défini par des paramètres spécifiques :

- **Note-On** : Déclenchement d'une note, associé à un pitch (hauteur) et une vitesse (intensité).
- **Note-Off** : Arrêt de la note précédemment déclenchée.
- **Pitch** : Valeur numérique correspondant à la hauteur de la note, par exemple 60 pour le Do central (C4).
- **Velocity** : Intensité sonore de la note, variant de 0 (silence) à 127 (volume maximal).

Ces informations permettent un contrôle des événements musicaux, facilitant leur traitement algorithmique dans un contexte de génération par intelligence artificielle.

Événement	Paramètres (exemple)	Description
Note-On	Pitch=60, Velocity=100, Channel=1, Time=0	Déclenchement d'une note (Do central) avec une vélocité de 100 sur le canal 1 au temps 0.
Note-Off	Pitch=60, Velocity=0, Channel=1, Time=480	Arrêt de la note (Do central) sur le canal 1 au temps 480.
Contrôleur MIDI (Control Change)	Control=7, Value=100, Channel=1, Time=200	Modification du contrôleur 7 (volume principal) à la valeur 100 sur le canal 1 au temps 200.
Pitch Bend	Bend=4096, Channel=1, Time=300	Variation de la hauteur (bend) sur le canal 1 au temps 300 (valeur centrale 4096 = pas de pitch bend).

TABLE 4 – Exemples d'événements MIDI courants et leurs représentations numériques

3 Modélisation par réseaux de neurones récurrents (RNN)

3.1 Architecture du modèle utilisé

3.1.1 Présentation détaillée de l'architecture LTMS

Le modèle développé repose sur l'architecture LTMS (*Local Time Multi-Scale*), une architecture spécifique basée sur des réseaux de neurones récurrents (RNN) utilisant des unités LSTM (*Long Short-Term Memory*). Ce choix architectural répond directement à la problématique centrale du projet : la génération cohérente de musique dans un style rock, nécessitant une gestion simultanée et coordonnée de plusieurs instruments.

Les unités LSTM sont une variante avancée des réseaux neuronaux récurrents classiques (cf. 13), spécialement conçues pour traiter des données séquentielles avec des dépendances temporelles longues. Le principe fondamental des unités LSTM repose sur une structure interne dotée de « portes » (*gates*) régulant le flux d'informations. Ces portes contrôlent sélectivement l'information à retenir ou à oublier, permettant ainsi de capturer efficacement des relations sur de longues séquences temporelles, tout en évitant les problèmes de disparition ou d'explosion du gradient caractéristiques des RNN simples. Concrètement, une unité LSTM se compose de trois portes principales :

- **Porte d'oubli (*forget gate*)** : Détermine quelle information historique supprimer de la mémoire interne de la cellule.
- **Porte d'entrée (*input gate*)** : Décide quelles nouvelles informations intégrer à la mémoire interne de la cellule.
- **Porte de sortie (*output gate*)** : Sélectionne quelles informations issues de la mémoire interne transmettre à l'état suivant ou en sortie.

Cette conception complexe assure une gestion dynamique et fine des informations contextuelles au fil du temps, essentielle pour modéliser des séquences musicales réalistes et cohérentes.

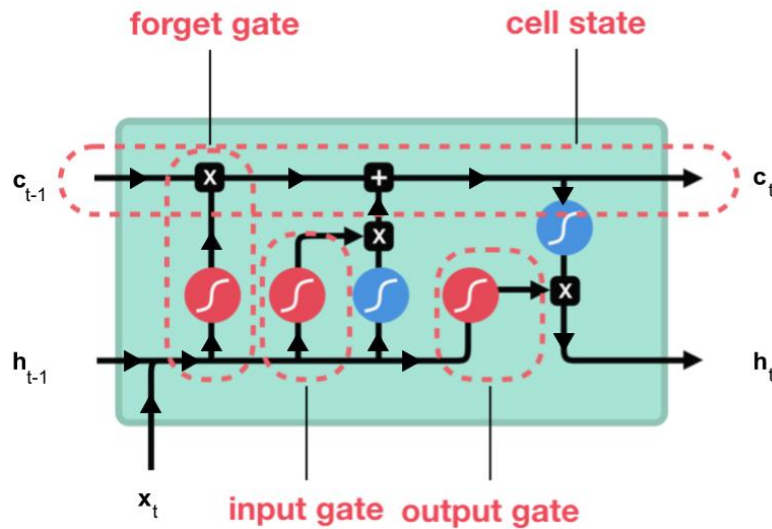


FIGURE 13 – Schéma détaillé du fonctionnement interne d'une unité LSTM

L'approche LTMS exploite pleinement cette capacité des unités LSTM en organisant les couches neuronales à différentes échelles temporelles. Ainsi, chaque instrument est traité séparément par des couches LSTM locales, dédiées à capturer les caractéristiques spécifiques de chaque canal instrumental (cf. 14). Par la suite, une couche LSTM globale agrège ces représentations locales, permettant une interaction efficace entre les différents instruments et assurant ainsi la cohérence harmonique, mélodique et rythmique de la génération finale (cf. 15).

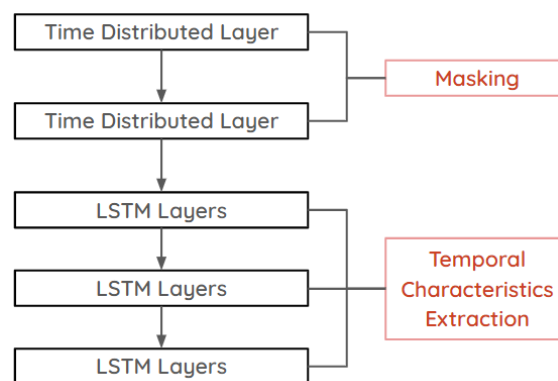


FIGURE 14 – Schéma de l'architecture de traitement local (pour chaque canal) des inputs

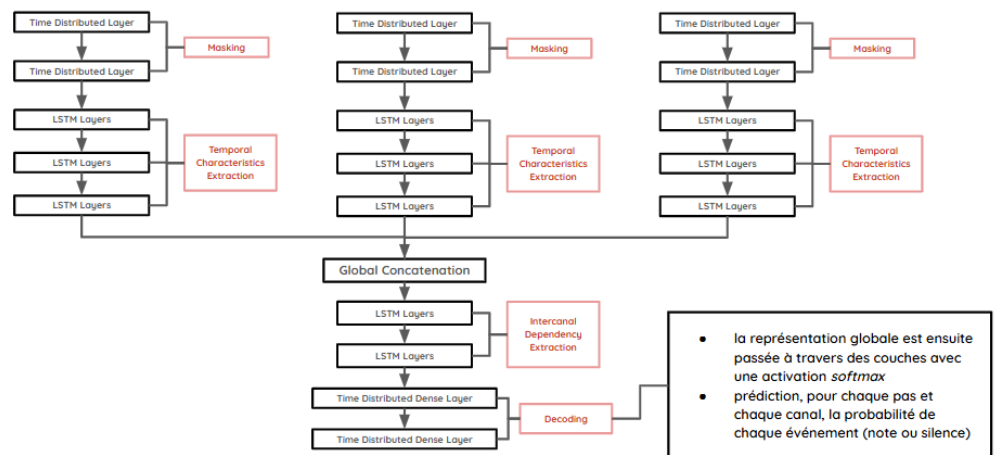


FIGURE 15 – Schéma de l'architecture de traitement global (multi-canal) des inputs

Cette structure hiérarchique garantit une gestion équilibrée des détails musicaux locaux tout en maintenant une cohérence globale indispensable à la qualité musicale des morceaux générés.

3.2 Implémentation technique

3.2.1 Environnement de développement et bibliothèques utilisées

L'implémentation technique du projet a été réalisée en langage Python, exploitant les bibliothèques d'apprentissage profond *TensorFlow* et *Keras*. Ces bibliothèques, largement reconnues dans la communauté scientifique et industrielle, offrent une grande flexibilité, efficacité computationnelle et facilité d'utilisation, particulièrement adaptées à la modélisation par réseaux de neurones récurrents.

Pour le lien avec le format MIDI, la bibliothèque *PrettyMidi* est utilisée pour l'encodage, et la bibliothèque *Mido* est utilisée pour la compatibilité de fichiers produits avec un lecteur de fichier MIDI.

L'interface graphique utilise le module *PyQt5* pour fonctionner.

3.2.2 Organisation du code source

Le code source a été organisé en différents scripts :

- **encodage.py** : Responsable de l'encodage des données musicales (fichiers MIDI) en représentations tensorielles exploitables par les couches LSTM.
- **decodage.py** : Fonctionnement inverse de **encodage.py**. Convertit les données générées par le modèle en fichiers MIDI utilisables pour l'écoute et l'analyse qualitative.
- **LSTM_units_Architecture.py** : Gère le processus d'entraînement du modèle, y compris le chargement et le traitement initial des données, l'entraînement proprement dit et la sauvegarde du modèle entraîné.
- **main.py** : Centralise l'exécution des différentes étapes du projet (encodage, entraînement, génération musicale, gestion des résultats).
- **gui.py** : Interface graphique développée avec le module *PyQt5* permettant de simplifier le choix des paramètres et l'interaction avec le modèle entraîné.

4 Méthode d'entraînement du modèle

4.1 Préparation des données

4.1.1 Sélection et description de la base de données MIDI

L'entraînement du modèle nécessite une base de données suffisamment représentative du style musical ciblé. Pour cela, une collection de fichiers MIDI issus du répertoire rock a été constituée. Cette sélection comprend des morceaux d'artistes reconnus tels que Radiohead, Nirvana, Pink Floyd ou The Beatles (certains des albums choisis en figure 16), garantissant à la fois variété rythmique et cohérence stylistique.



FIGURE 16 – Albums dont sont extraits les titres utilisés

4.1.2 Méthodes de prétraitement et encodage

Avant d'être injectées dans le modèle, les données MIDI sont transformées en représentations numériques exploitables grâce au fichier `encodage.py`. Le prétraitement est divisé en plusieurs étapes :

1. **Classification des instruments** : Les instruments présents dans le morceau (guitares, piano, percussions, bois...) sont classifiés par type et limités dans leur nombre. Chaque instrument est assigné à un numéro de canal selon son type, comme présenté dans le tableau .
2. **Encodage des événements** : Le morceau est discrétisé temporellement, et chaque événement musical est converti en vecteur one-hot.

Canal	Instrument	Plage de notes MIDI
0	Guitar 1	40 – 88
1	Guitar 2	40 – 88
2	Guitar 3	40 – 88
3	Guitar 4	40 – 88
4	Bass	28 – 52
5	Keyboard 1	21 – 108
6	Keyboard 2	21 – 108
7	Keyboard 3	21 – 108
8	Keyboard 4	21 – 108
9	Drums	35 – 81
10	Accoustic	40 – 88
11	Reed 1	49 – 92
12	Flute	60 – 96
13	Reed 2	50 – 94
14	Trumpet	55 – 82
15	Special	Variable

TABLE 5 – Répartition des instruments par canal et plages de notes MIDI

Chaque morceau de musique est encodé selon une structure définie par quatre dimensions : nombre de pas temporels, nombre de canaux, nombre de notes simultanées, dimension de l'encodage one-hot. Une représentation schématique avec des listes imbriquées est montrée en figure 17.

D'abord, le morceau est une série temporelle des notes jouées par chaque instrument. Le temps est discrétisé en douzièmes de noires (la durée des pas de temps s'adapte donc au tempo) pour pouvoir capter des notes suffisamment courtes. Les notes plus longues sont donc découpées en plusieurs notes qui se poursuivent.

Ensuite, pour chaque pas de temps, pour chaque instrument (chacun des 16 canaux), on stocke la liste des notes jouées à cet instant. Si rien n'est joué, on note un silence 's'. Si une note débute, on note son pitch comme en tick 2 sur le schéma : '55'. Si une note continue parce que la discrétisation temporelle l'a découpée, on note son pitch avec la lettre 'p' pour "prolongation", comme en tick 3 : '55p'.

Enfin, puisqu'on connaît pour chaque classe d'instruments la plage de notes possibles (voir sur le tableau 5), chaque note est convertie en one-hot-vector suivant l'instrument qui la joue. Pour un instrument qui peut jouer nb_{pitch} notes différentes, la taille du vecteur est $2 \times nb_{pitch} + 1$ pour encoder les prolongations et les silences. Cette étape n'est pas représentée sur le schéma par souci de lisibilité.

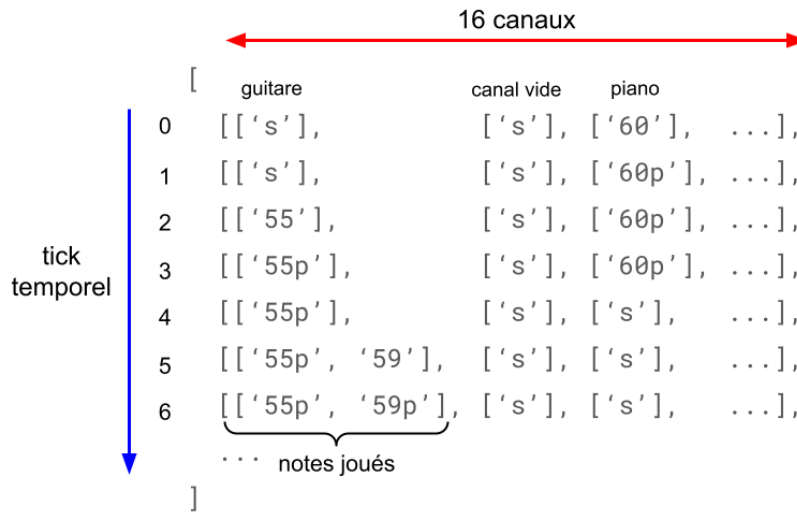


FIGURE 17 – Exemple d'encodage de morceau de musique

4.1.3 Représentation finale des données pour l'entraînement

Les tenseurs obtenus à l'issue du prétraitement servent d'entrées (X_{train}) et de cibles (Y_{train}) pour le modèle. À chaque pas de temps, le réseau apprend à prédire la séquence suivante à partir du contexte déjà généré. Cette structure autoregressive est essentielle pour garantir une continuité musicale fluide lors de la génération finale.

4.2 Paramètres et hyperparamètres d'entraînement

Plusieurs hyperparamètres déterminent le comportement et la performance du modèle. Ils peuvent être ajustés dynamiquement via l'interface graphique `gui.py` :

- **Epochs** : Nombre de cycles complets d'apprentissage sur l'ensemble des données. Une valeur élevée favorise une meilleure convergence, au risque de surapprentissage.
- **Batch Size** : Nombre de séquences traitées en parallèle. Ce paramètre influence la stabilité du gradient et la consommation mémoire.
- **Température** : Intervient uniquement lors de la génération ; elle ajuste la distribution de probabilité des notes. Une température faible engendre une sortie plus prévisible, une température élevée favorise la diversité.
- **Nombre de cycles** : Détermine la longueur des morceaux générés, chaque cycle correspondant à un segment temporel dans la structure du morceau final.

4.3 Processus détaillé d'entraînement

L'entraînement du modèle se déroule en plusieurs étapes (cf. figure 18) :

1. **Chargement des données** : Les tenseurs encodés pour chaque instrument sont importés en mémoire.
2. **Construction du modèle** : L'architecture LTMS est instanciée avec les dimensions et paramètres fournis (nombre d'instruments, dimensions des encodages).
3. **Compilation** : Le modèle est compilé avec une fonction de perte de type `categorical_crossentropy` et l'optimiseur Adam.
4. **Entraînement** : La méthode `model.fit()` de Keras est utilisée pour entraîner le réseau. Des callbacks permettent de suivre l'évolution de la performance et d'enregistrer des checkpoints intermédiaires.
5. **Sauvegarde du modèle** : Une fois l'apprentissage terminé, les poids du modèle sont enregistrés pour être utilisés lors de la phase de génération.

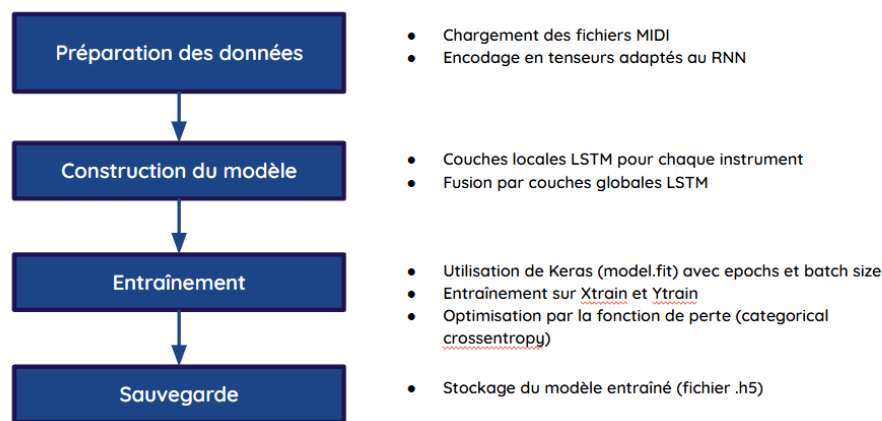


FIGURE 18 – Schéma descriptif de l'entraînement du modèle

5 Génération de musique par le modèle entraîné

5.1 Méthodologie de génération musicale

Une fois entraîné, le modèle peut générer de nouveaux morceaux de musique selon un fonctionnement *autoregressif*. Cela signifie que la génération se fait pas à pas : à chaque itération, le modèle prédit les événements musicaux à venir en se basant uniquement sur ce qu'il a déjà généré jusqu'à présent.

Le processus commence par la création d'un **seed initial**, une courte séquence générée aléatoirement pour chaque instrument, ou basée sur un début de morceau. Ce seed constitue le contexte de départ, comparable à une amorce musicale. À partir de celui-ci, le modèle prédit les événements (notes, silences, prolongations) du pas de temps suivant pour chaque canal, puis incorpore cette prédiction à la séquence et recommence.

Ce mécanisme se poursuit jusqu'à produire une séquence complète. Afin de structurer le morceau et éviter une monotonie excessive, la génération se fait par **cycles musicaux**, unités rythmiques de durée fixe (par exemple 64 ou 128 pas de temps). Le nombre total de cycles à générer est un paramètre défini par l'utilisateur via l'interface graphique.

Entre ces cycles, des **transitions** plus courtes peuvent être insérées de manière aléatoire. Ces transitions permettent d'apporter des variations et de simuler une certaine dynamique structurelle dans la composition, en imitant par exemple l'effet de ponts ou de breaks dans un morceau de rock.

Un paramètre important dans le processus est la **température** (ou "temperature"), qui intervient au moment du *sampling*, c'est-à-dire du choix de la prochaine note parmi toutes les options possibles. Une température basse (proche de 0.1) favorise les choix très probables (comportement plus déterministe, donc plus répétitif), tandis qu'une température plus élevée (autour de 1.0 ou plus) rend les choix plus variés, voire imprévisibles.

5.2 Décodage des résultats en fichiers musicaux

La sortie du modèle est un tenseur à 4 dimensions représentant la séquence générée au même format qu'en entrée (voir la section dédiée 4.1.2).

La différence est que certaines notes vides ont été ajoutées pour que les dimensions du tenseur soient homogènes : par exemple on fait en sorte que tous les événements soient des accords de 4 notes, quitte à ajouter des notes 's'. Ces notes sont filtrées.

Chaque vecteur one-hot encode une note jouée à un instant donné. Le décodage de ce tenseur en musique exploitable s'effectue en plusieurs opérations essentielles :

1. **Décodage des vecteurs one-hot** : Chaque vecteur est transformé en note identifiable, avec gestion des silences et des prolongations de note (notées "p" dans l'encodage).
2. **Regroupement par instrument** : Les notes sont réorganisées selon les 16 canaux instrumentaux (guitares, basse, batterie, claviers, etc.), permettant une interprétation distincte de chaque piste.
3. **Reconstruction temporelle** : Chaque note est assignée à un instant de début et une durée précise, en tenant compte de sa position dans la séquence et des prolongations. Pour des soucis de facilité d'encodage, la vitesse de toutes les notes reconstituées est la même.
4. **Création du fichier MIDI** : Un objet `PrettyMIDI` est instancié, puis complété avec toutes les pistes instrumentales contenant les notes précédemment reconstituées.
5. **Correction du tempo** : Afin de garantir une lecture correcte dans les éditeurs MIDI comme MuseScore, un méta-message de tempo est injecté dans le fichier.

Le fichier final, de type `.mid`, peut alors être ouvert avec n'importe quel logiciel compatible (MuseScore, Logic Pro, FL Studio...), permettant une écoute directe ou une édition manuelle.

6 Résultats et analyse

6.1 Génération de progressions harmoniques avec LSTM

6.1.1 Principe et tests réalisés

Avant de procéder à la génération de morceaux entiers, des expérimentations intermédiaires ont été menées sur un sous-problème plus simple : la génération de **progressions d'accords** (suite d'accords harmoniques) à l'aide d'un réseau LSTM monophonique.

Le but de ces tests est d'évaluer la capacité du modèle à reproduire des successions cohérentes d'accords à partir d'une base de données de séquences existantes, en utilisant des représentations simplifiées : chaque accord est encodé sous forme d'un identifiant unique, et les progressions sont considérées comme des chaînes de longueur fixe.

Le modèle est entraîné à prédire l'accord suivant à partir d'une séquence d'entrée partielle. Il s'agit donc d'un problème de prédiction séquentielle classique. Une fois l'entraînement effectué, le modèle peut générer des progressions entières en partant d'un accord initial choisi au hasard.

6.1.2 Évaluation qualitative des séquences générées

Plusieurs séries de tests ont été réalisées avec des températures différentes (0.5, 1.0, 1.5) pour observer les variations de style et de diversité. Voici quelques observations qualitatives :

- À température basse, les progressions sont stables, souvent répétitives, et très proches des structures les plus fréquemment rencontrées dans les données d'entraînement (ex : I–V–vi–IV).
- À température moyenne, les séquences gagnent en diversité tout en conservant des transitions harmoniques crédibles.
- À température élevée, des successions inattendues ou dissonantes peuvent apparaître, parfois proches d'un style expérimental ou “free jazz”.

Les progressions générées ont été exportées sous forme de fichiers MIDI, puis écoutées via un séquenceur numérique. Plusieurs séquences s'avèrent exploitables telles quelles comme base harmonique pour des improvisations ou compositions simples.

Par exemple, les progressions suivantes :

- Isus4 I II IV I V V IV
- IV V I II V VI I V II I VI IV V I V I
- VI7 VII I I I I VII I V I III I VII I IV VII

— VI_{sus}4 IIIM IV IV I V I V I IV I I I II I I

6.2 Génération de morceaux complets

6.2.1 Présentation de l'interface graphique utilisateur

L'interface graphique permet de gérer l'entraînement du modèle. Il suffit d'exécuter le script `gui.py` pour qu'elle s'ouvre (cf. 19).

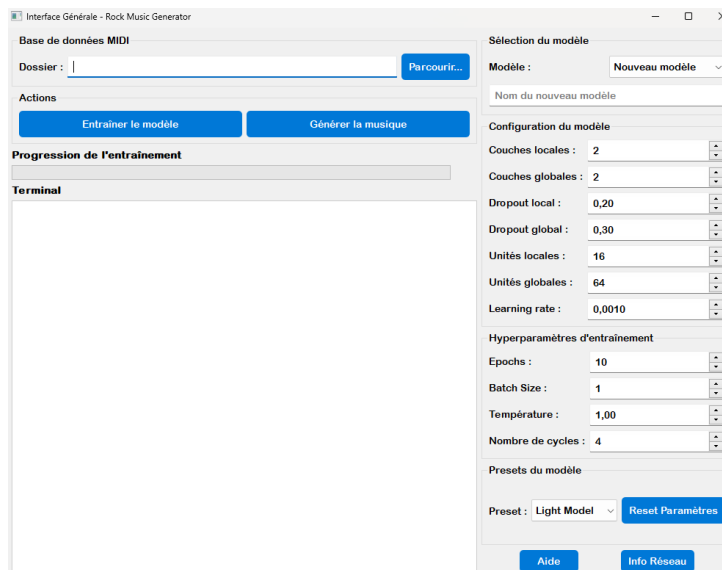


FIGURE 19 – Interface graphique au démarrage

Ensuite, l'utilisateur peut sélectionner un dossier contenant les fichiers d'entraînement. Tous les fichiers `.mid` du dossier choisi seront utilisés.

À droite de l'interface, un menu déroulant permet de choisir un modèle déjà existant. Le "Light Model" est un modèle réduit, celui utilisé pour le test sur "Twist and shout" (voir en section 6.2.2). Le "LTMS Model" est plus grand, il n'a pas pu être utilisé pour les tests à cause du manque de puissance de nos machines. En dessous de ce menu, les hyper-paramètres et la structure du réseau sont affichés et modifiables. De plus, un menu d'aide rappelle le sens des différents paramètres (cf. 20).

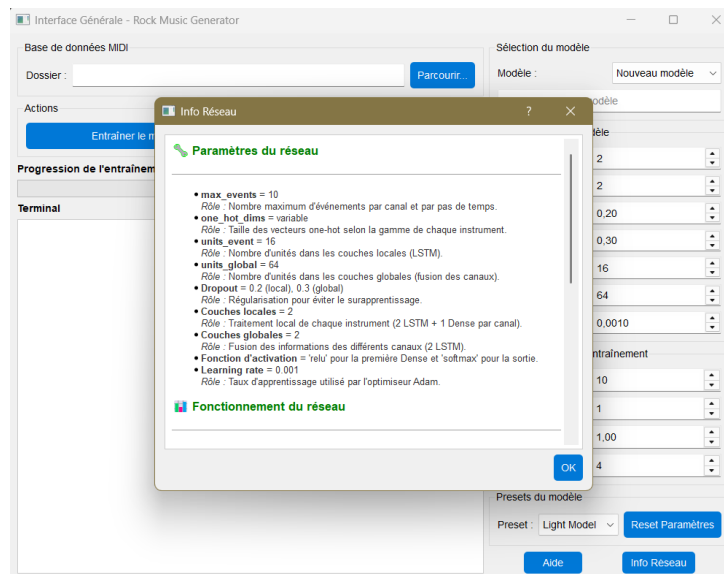


FIGURE 20 – Menu d'aide

Après un clic sur “Entraînement du modèle” ou “Générer la musique”, les messages d’avancement sont affichés dans la console (cf. figure 21).

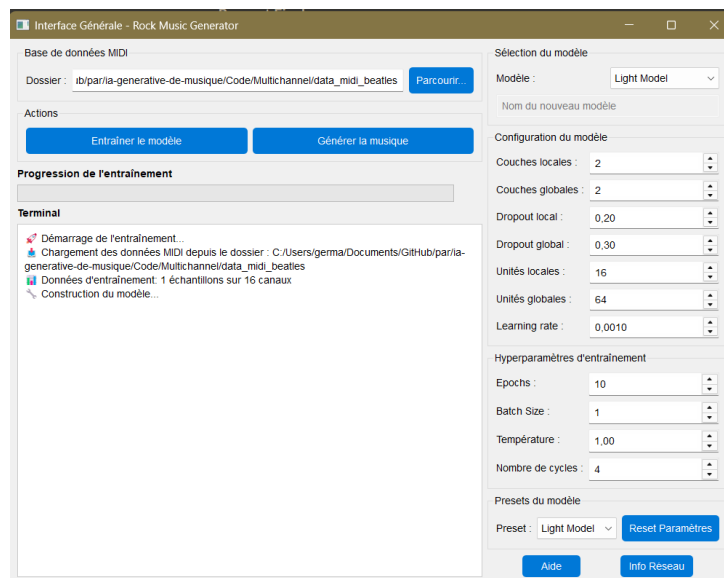


FIGURE 21 – Message pendant l’entraînement du modèle

6.2.2 Analyse qualitative des résultats

Tests simplifiés Ces résultats ont été produits en entraînant le réseau sur une entrée très simplifiée (cf. 22). Le but est de réduire la taille de la séquence à analyser, de la séquence à produire et la diversité des notes. Le nombre de neurones est réduit, ainsi que le nombre maximal d’époques d’entraînement.

Le morceau utilisé pour l’entraînement est constitué de seulement 1 canal, et 3 notes de piano. La séquence découpée en douzièmes de noires ne fait donc que 60 éléments. Les notes sont trois blanches, et suffisamment espacées.



FIGURE 22 – Morceau de test affiché avec MuseScore

Après génération, la séquence produite est analysée dans MuseScore (cf. figure 23). D’abord, observons que les notes jouées ne sont jamais prolongées, cela donne un rythme trop rapide à la séquence. Et aussi, tous les accords sont identiques, constitués du maximum d’événements simultanés possibles (ici 4).



FIGURE 23 – Résultat du test simplifié, affiché dans MuseScore

Cela montre un fort sous-apprentissage puisque la possibilité de différentes notes n’est même pas utilisée pour générer la sortie. La couche softmax pour choisir la note à générer fait un choix parmi une gamme large, mais a été entraînée très peu de temps (5 époques), sur très peu de données (1 fichier MIDI). En résultat, le modèle prédit toujours les mêmes notes, très rapides.

Morceau unique en entrée Ce deuxième type de test est réalisé avec un morceau complet en entrée. Ici, le morceau est “Twist and shout” des Beatles, déjà présenté en figure 8. Le nombre total de notes est 7669. Cela reste un exemple simple puisqu’il n’y a que 5 canaux : une basse, un piano, deux guitares et une batterie.

Les sorties sont générées partir d’une petite séquence d’amorçage aléatoire, cela garantit de la variabilité dans le fichier généré.

Une des sorties produites est en figure 24. Comme pour les tests précédents, il n’y a pas de notes prolongées, et beaucoup de notes sont jouées en même temps. Cela rend ce fichier difficile à écouter. Cependant, les notes jouées ne sont pas toutes identiques. C’est même l’inverse, trop peu se répètent pour faire émerger un rythme ou une mélodie. Un paramètre de température mal adapté est probablement responsable.

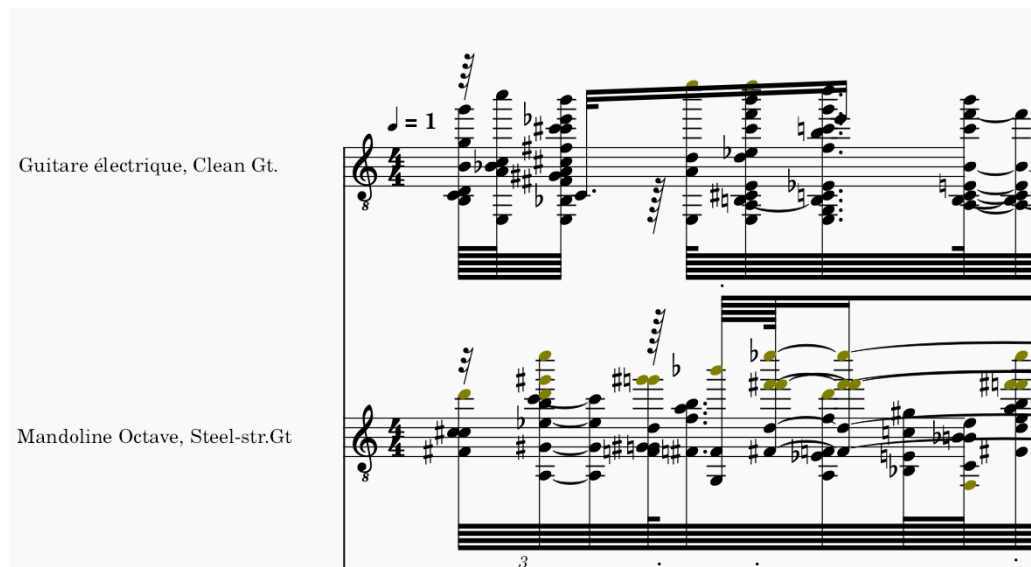


FIGURE 24 – Résultat de l’entraînement “Twist and shout”

6.3 Limites et perspectives

Limites de l’encodage Pour l’encodage de la musique vers le format de tenseur pour l’entrée du réseau de neurones, notons qu’il y a quelques simplifications utilisées :

- La limite du nombre de canaux peut imposer d’en ignorer certains, et cela nous contraint de sélectionner des données d’apprentissage avec les mêmes instruments.
- La représentation en one-hot-vectors implique que l’IA produira seulement des notes déjà rencontrées, il n’est pas possible de produire de la musique micro-tonale par exemple.
- Le découpage temporel pourrait passer à côté de notes trop courtes.
- Certains aspects de la musique en entrée sont présents dans le fichier MIDI, mais ne sont pas gardés dans le tenseur. Par exemple, la vélocité des notes ou les changements de tempo.

Toutefois, les fichiers utilisés pour les tests sont relativement simples, donc ce ne sont pas des aspects très limitants pour produire de la musique cohérente.

Entraînement du réseau Les tests présentés montrent que l’entraînement n’est pas du tout satisfaisant. Dans l’hypothèse qu’il est insuffisant, une amélioration pourrait être d’augmenter le nombre d’époques d’entraînement, ou le nombre de paramètres en complexifiant le réseau.

De plus, la limitation de la puissance des machines utilisées n’a pas permis de pousser très loin le nombre d’entrées utilisées, et la complexité des sorties.

Méthode de génération Au vu des résultats, les perspectives d’améliorations sont plus grandes du côté de la méthode de génération.

La génération actuelle se fait tick par tick, mais pour les 16 canaux en même temps. À chaque étape les 16 canaux génèrent un événement basé sur les séquences déjà générées

par les 16 canaux. Cela permet facilement une dépendance inter-canaux (le rythme de la batterie va influencer la mélodie de la guitare).

Mais un autre fonctionnement serait de générer la séquence indépendamment par chaque canal. L'étape délicate devient la mise en commun des instruments, qui semble dure à automatiser.

Conclusion

Pour rappel, ce projet avait pour objectif de concevoir un système de génération automatique de musique dans le style rock à l'aide d'un réseau de neurones récurrent. Le cœur de la démarche reposait sur l'architecture LTMS (Local Time Multi-Scale), capable de traiter simultanément plusieurs instruments répartis sur 16 canaux, et d'assurer une cohérence harmonique et rythmique à l'échelle globale.

La première étape a consisté en traitement de fichiers MIDI. Un travail conséquent a été mené pour encadrer l'encodage symbolique automatique des événements musicaux, leur organisation par instrument, et leur transformation en tenseurs compatibles avec l'apprentissage supervisé. Après un état de l'art des méthodes possibles l'architecture LTMS retenue a ensuite été implémentée en Python. L'environnement a été complété par une interface graphique interactive permettant à l'utilisateur d'entraîner un modèle, d'en ajuster les paramètres, et de générer de nouvelles séquences musicales.

Plusieurs expériences de génération ont été réalisées, à partir de données simples ou de morceaux complets. Bien que les résultats montrent certaines limites (notamment un sous-apprentissage, une redondance excessive dans les séquences générées ou l'absence de notes longues), le système peut être amélioré vers un modèle plus ambitieux. Les écarts constatés s'expliquent par un temps d'entraînement limité, une puissance de calcul restreinte, et une base de données volontairement réduite.

En termes d'apports techniques, ce projet a permis de confronter des connaissances théoriques en deep learning à un cas concret. Il a mobilisé des compétences, en manipulation du format MIDI, et en conception de réseaux de neurones.

Bibliographie

- [1] Filippo CARNOVALINI et Antonio RODÀ. “Computational Creativity and Music Generation Systems : An Introduction to the State of the Art”. In : *Frontiers in Artificial Intelligence* 3 (2020), p. 14.
- [2] *AIVA, the AI Music Generation Assistant*. URL : <https://www.aiva.ai/> (visité le 08/04/2025).
- [3] Yueyue ZHU et al. *A Survey of AI Music Generation Tools and Models*. 24 août 2023. DOI : 10.48550/arXiv.2308.12982. URL : <http://arxiv.org/abs/2308.12982> (visité le 17/12/2024). Prépubl.
- [4] Yichao ZHOU et al. *BandNet : A Neural Network-based, Multi-Instrument Beatles-Style MIDI Music Composition Machine*. 18 déc. 2018. DOI : 10.48550/arXiv.1812.07126. URL : <http://arxiv.org/abs/1812.07126> (visité le 17/12/2024). Prépubl.
- [5] *Réseau de neurones artificiels*. In : *Wikipédia*. 6 jan. 2025.
- [6] *Neurone*. In : *Wikipédia*. 29 déc. 2024.
- [7] *OpenAI’s New Language Generator GPT-3 Is Shockingly Good—and Completely Mindless*. MIT Technology Review. URL : <https://www.technologyreview.com/2020/07/20/1005454/openai-machine-learning-language-generator-gpt-3-nlp/> (visité le 07/01/2025).
- [8] Jean-Pierre BRIOT. “From Artificial Neural Networks to Deep Learning for Music Generation : History, Concepts and Trends”. In : *Neural Computing and Applications* 33.1 (jan. 2021), p. 39-65. DOI : 10.1007/s00521-020-05399-0.
- [9] Jiaxing YU et al. “Suno : Potential, Prospects, and Trends”. In : *Frontiers of Information Technology & Electronic Engineering* 25.7 (juill. 2024), p. 1025-1030. DOI : 10.1631/FITEE.2400299.
- [10] *Rppmf [Download] - Novembre 2024*. URL : <https://rppmf.com/archives.htm> (visité le 07/01/2025).
- [11] *The MAESTRO Dataset*. Magenta. 29 oct. 2018. URL : <https://magenta.tensorflow.org/datasets/maestro> (visité le 07/01/2025).
- [12] *URMP Dataset*. URL : <https://labsites.rochester.edu/air/projects/URMP.html> (visité le 07/01/2025).